

Banking Management System

Professional Project Documentation

A source-grounded technical and functional reference for the Java 26 / Spring Boot application

Document	Project Documentation
Project	BankApplication
Artifact version	0.0.1-SNAPSHOT
Runtime target	Java 26
Framework	Spring Boot 4.1.1
Database	MySQL
Migration engine	Flyway
Primary API namespace	/api/v1
Document date	16 September 2026

Documentation basis: the supplied BankApplication source archive and the supplied reference PDF, “Java Final Project Documentation”. The reference PDF provides the baseline documentation organization; the project archive is treated as authoritative for implemented project details.

Table of Contents

- 1. Executive Summary
- 2. Project Objectives and Scope
- 3. Technology Stack
- 4. System Architecture
- 5. Package and Source Structure
- 6. Functional Modules
- 7. Roles and Authorization
- 8. Security Architecture
- 9. Business Flow Diagrams
- 10. Data Model and Database Design
- 11. Database Relationships
- 12. REST API Reference
- 13. Validation and Error Handling
- 14. Transaction Integrity and Financial Controls
- 15. Idempotency Design
- 16. Auditing and Observability
- 17. Caching
- 18. Scheduling and Maintenance
- 19. Notifications and Domain Events
- 20. File and Document Management
- 21. Reporting
- 22. Configuration and Environments
- 23. Database Migration History
- 24. Testing Assets
- 25. Build and Dependency Management
- 26. API Documentation and Operational Interfaces
- 27. Professional Deployment Considerations
- 28. Implementation Notes and Source Boundaries
- 29. Conclusion
- Appendix A. Key Source Components

1. Executive Summary

BankApplication is a modular banking management application implemented with Java 26 and Spring Boot 4.1.1. The source archive contains a layered, feature-oriented backend with REST controllers, DTOs, MapStruct mappers, JPA entities/repositories, service interfaces and implementations, centralized validation and exception handling, JWT-based REST security, audit logging, idempotency protection, asynchronous domain events, scheduled maintenance, caching, file storage, reporting, and Flyway database migrations.

The application is organized around core banking domains—users, customers, employees, accounts, transactions, loans, repayments and investments—with supporting operational domains for notifications, audit, reporting, documents, refresh tokens and idempotency. Financial amounts are represented with BigDecimal in the domain entities and database schema uses DECIMAL precision for monetary columns.

The source archive contains 238 Java source files, 3 test classes, 18 Flyway migration scripts. These counts are derived from the supplied project archive.

Document Scope

- System purpose and architectural boundaries
- Functional modules and lifecycle flows
- REST API surface and authorization model
- Relational data model and migration strategy
- Cross-cutting concerns: security, audit, idempotency, caching, events and scheduling
- Configuration profiles, observability and operational behavior
- Testing assets and implementation notes visible in the supplied source archive

2. Project Objectives and Scope

This documentation describes a finance-management style application around customers, accounts, transactions, loans and investments. BankApplication extends that baseline into a broader banking domain with identity, role separation, lifecycle controls, transaction semantics, loan disbursement and repayment, investment settlement, operational notifications, audit trails, idempotency and reporting.

Primary objectives evidenced by the implementation: provide secured REST APIs for banking operations; maintain relational consistency through JPA and Flyway; separate transport, business and persistence concerns; protect sensitive operations through role and resource authorization; record operational actions; prevent duplicate mutating requests; and support operational maintenance through scheduled jobs.

3. Technology Stack

Technology / Component	Implementation evidenced in project	Purpose
Java	26	Application language and runtime target.
Spring Boot	4.1.1	Application framework and dependency platform.
Spring Data JPA	Spring Boot dependency	ORM and repository abstraction.
MySQL	mysql-connector-j	Relational persistence.
Flyway	spring-boot-starter-flyway	Versioned schema migrations V1–V18.
Spring Security	Security + OAuth2 Resource Server	REST JWT authentication and MVC form/session security.
MapStruct	1.6.3	DTO/entity mapping.
Lombok	Project dependency	Boilerplate reduction.
Caffeine	Cache provider	Application-level caching.
Spring AOP	spring-boot-starter-aspectj	Logging, performance, audit and idempotency aspects.
Springdoc/OpenAPI	Configured in application.yml	OpenAPI specification and Swagger UI for /api/v1.
Testcontainers	1.20.6	MySQL/JUnit integration-test support.
Apache Tika / Spring AI Tika reader	Tika 3.3.1 / Spring AI 2.0.1	Document-reading related dependency support.

4. System Architecture

The implementation follows a feature-oriented layered architecture. Each major business domain is grouped under its own package and typically contains controller, DTO, entity, mapper, repository and service layers. Cross-cutting infrastructure is centralized under common, config, security, aspect, event and scheduler packages.

Architecture Flow

Client / Consumer	Security Boundary	REST Controller	Service Layer	Repository Layer	MySQL	Events & Async
REST client, browser or integrated caller	JWT REST chain or MVC session/form chain	HTTP contract, validation and authorization	Transactional business rules	Spring Data JPA / custom reporting queries	Relational state managed by Flyway	Notifications and audit after transaction completion
Client / Consumer → Security Boundary → REST Controller → Service Layer → Repository Layer → MySQL → Events & Async						

Architectural Principles

- REST endpoints are versioned under /api/v1.
- DTOs separate request/response contracts from persistence entities.
- Service interfaces and implementations provide a consistent application-service boundary.
- @Transactional is used in business services where multi-step state changes must remain atomic.
- JPA auditing provides created/updated actor metadata for selected mutable entities.
- open-in-view is disabled, keeping persistence concerns explicit.
- Database schema is validated by Hibernate while Flyway owns schema evolution.

5. Package and Source Structure

The project contains 120 Java packages under com.example.bank. The following high-level structure captures the architectural organization without reproducing the entire source tree.

com.example.bank	
└─ account/	accounts, DTOs, mapper, repository, service, REST
└─ admin/	administrator lifecycle and admin account controls
└─ audit/	persistent audit log API and service
└─ auth/	login, refresh, password change, logout
└─ common/	audit, constants, enums, exceptions, responses, utilities
└─ config/	security, JWT, caching, scheduling, MVC, OpenAPI, async
└─ customer/	customer lifecycle and authorization
└─ employee/	employee lifecycle
└─ event/	domain events and listeners
└─ file/	document metadata and local file storage
└─ idempotency/	duplicate-request protection
└─ investment/	investment lifecycle and settlement
└─ investmentperformance/	performance snapshots
└─ loan/	loan lifecycle and disbursement
└─ loanrepayment/	repayment lifecycle
└─ notification/	user notifications
└─ reporting/	paged reporting queries and DTOs
└─ scheduler/	scheduled maintenance/business checks
└─ security/	JWT, authentication, resource authorization
└─ transaction/	deposit, withdrawal, transfer, reversal
└─ user/	user lifecycle

The source archive also contains resources under src/main/resources, including profile-based YAML configuration, Flyway migrations.

6. Functional Modules

Authentication & Identity

User authentication, JWT access tokens, refresh-token lifecycle, password change and logout.

Users

Administrative user lifecycle and user lookup.

Customers

Customer profile lifecycle, activation/deactivation and user association.

Employees

Employee profile and administrative lifecycle.

Accounts

Account creation, lookup and lifecycle controls including block, activate, deactivate and close.

Transactions

Deposits, withdrawals, transfers, transaction history and reversal.

Loans

Loan application, eligibility, approval/rejection, completion and disbursement.

Loan Repayments

Repayment processing, repayment history and outstanding-amount support.

Investments

Investment creation, maturity and settlement into an account.

Investment Performance

Performance snapshots and latest-performance retrieval.

Files & Documents

Customer document metadata, local storage, upload/download and deletion.

Notifications

User notifications generated from domain events and read-state management.

Audit

Service-level audit events and persistent audit-log search/retrieval.

Reporting

Paged customer, account, transaction, loan and investment reports.

Idempotency

Duplicate-request protection for mutating REST operations.

Scheduling & Maintenance

Investment maturity, loan monitoring and maintenance cleanup jobs.

7. Roles and Authorization

Role	Documented / implemented access areas
ADMIN	Administrative REST endpoints; user, admin, employee and customer lifecycle operations; account administration; reporting; audit-log access; investment/loan/transaction operational controls where method authorization permits.
EMPLOYEE	Employee/customer/account operational functions and operational reporting; loan approval/rejection/completion/disbursement and other method-authorized banking operations.
CUSTOMER	Customer-scoped account access, transfers and other self-service operations; customer-scoped loans and investments; notifications and customer documents subject to resource authorization.

Security is enforced at two levels: the REST security filter chain requires authentication for `/api/**` except explicitly public authentication endpoints, and `@PreAuthorize` expressions add role/resource-level authorization at controller methods. Resource authorization services exist for customers, accounts, transactions, loans, investments, loan repayments, investment performance and file metadata.

8. Security Architecture

REST API Security

The REST security chain is ordered first and matches `/api/**`. It uses stateless sessions, disables CSRF for the bearer-token API, and configures Spring Security OAuth2 Resource Server JWT processing. Public authentication endpoints are supplied through `SecurityConstants.PUBLIC_API_ENDPOINTS`; administrative REST paths under `/api/v1/admin/**` require ADMIN.

JWTs are issued using an RSA key pair. `JwtConfig` loads Base64-encoded private/public keys, creates a `Nimbus JwtEncoder` and `JwtDecoder`, and validates the configured issuer. `JwtTokenService` creates RS256-signed access tokens containing issuer, subject, issued/expiry times, `userId`, role and authorities.

Password and Token Controls

- Passwords are encoded with `BCryptPasswordEncoder`.
- Refresh tokens are persisted as token hashes, with expiry, revocation and replacement relationships.
- Refresh-token cleanup is scheduled.
- REST error handling separates unauthenticated (401) and unauthorized (403) responses through dedicated handlers.
- `SecurityAuditAspect` provides a separate security-focused AOP concern.

9. Business Flow Diagrams

9.1 Transaction Transfer Flow

Client	Security	Idempotency	Transaction Service	Database	Events	Notification
POST /transfer	JWT + authorization	Key + request hash	Validate + debit/credit	Accounts + ledger	TransactionCompleted	AFTER_COMMIT async
Client → Security → Idempotency → Transaction Service → Database → Events → Notification						

A transfer is exposed as `POST /api/v1/transactions/transfer`. The transaction service owns the financial operation; the implementation contains account/transaction authorization services, idempotency protection for mutating operations, transactional service methods and a `TransactionCompletedEvent` consumed asynchronously after commit for notifications.

9.2 Loan Lifecycle

Application	Eligibility	Review	Approved / Rejected	Disbursement	Active / Completed	Repayment
Loan request	Eligibility check	ADMIN / EMPLOYEE	Decision + reason	Credit selected account	Lifecycle	Loan-linked repayment
Application → Eligibility → Review → Approved / Rejected → Disbursement → Active / Completed → Repayment						

9.3 Investment Maturity and Settlement

Investment Active	Maturity Check	Matured	Settlement	Transaction Ledger	Notification
Customer/account-linked	Scheduled processing	Status transition	Settlement reference	Settlement transaction	Maturity event
Investment Active → Maturity Check → Matured → Settlement → Transaction Ledger → Notification					

9.4 Event-Driven Operational Flow

Transactional Service	Application Event	AFTER_COMMIT	Async Executor	Notification / Audit
Business operation	Domain event	Transaction complete	bankTaskExecutor	Persist record
Transactional Service → Application Event → AFTER_COMMIT → Async Executor → Notification / Audit				

10. Data Model and Database Design

The database is managed through 18 Flyway migrations. The schema uses BIGINT identity keys, explicit unique constraints, foreign keys, CHECK constraints and indexes. Financial values are stored as DECIMAL(19,4) in the main monetary tables, matching the use of BigDecimal in the entity model.

Entity	Principal fields	Purpose
User	id, username, passwordHash, role, status, createdAt, updatedAt, createdBy, updatedBy	Authentication identity and role/status.
Customer	id, user, name, email, phone, address, status, audit timestamps/actors	Customer profile linked one-to-one to a user.
Employee	id, user, employeeCode, name, email, phone, department, designation, audit timestamps/actors	Employee profile linked one-to-one to a user.
Account	id, customer, accountNumber, accountType, balance, status, audit timestamps/actors, version	Customer deposit account with optimistic-locking version.
Transaction	id, account, transactionReference, transactionType, amount, transactionDate, description, counterpartyAccount, transactionStatus, createdAt	Financial movement recorded against an account.
Loan	id, customer, loanType, principalAmount, interestRate, startDate, endDate, status, rejectionReason, disbursement fields, audit/version	Loan lifecycle and disbursement state.
LoanRepayment	id, loan, account, transactionReference, amount, repaymentDate, status, description, createdAt, version	Repayment record tied to a loan and account.
Investment	id, customer, account, investmentType, amount, startDate, maturityDate, status, description, settlement fields, audit/version	Investment held against a customer/account.
InvestmentPerformance	id, investment, performanceDate, investedAmount, currentValue, profitLossAmount, returnPercentage, notes, createdAt	Dated investment performance snapshot.
FileMetadata	id, customer, uploadedByUser, documentType, original/stored file names, path, content type, size, checksum, status, audit/version	Document metadata and local-storage reference.
Notification	id, user, eventId, notificationType, title, message, referenceType, referenceId, readAt, createdAt	User-facing event notification.
AuditLog	id, eventId, userId, username, role, action, entityType/id, service/method, HTTP context, success/error, timestamps	Persistent service-level audit record.
ApiIdempotencyKey	id, user, idempotencyKey, requestHash, status, HTTP status, response body, lifecycle timestamps	Per-user duplicate-request protection.
RefreshToken	id, user, tokenHash, expiresAt, createdAt, revokedAt, replacedByToken	Refresh-token rotation/revocation state.

11. Database Relationships

Relationship	Cardinality / rule
User → Customer	Customer has a unique user_id; one customer profile per user.
User → Employee	Employee has a unique user_id; one employee profile per user.
Customer → Account	One customer can own multiple accounts.
Account → Transaction	One account can have multiple transaction records.
Customer → Loan	One customer can have multiple loans.
Loan → LoanRepayment	One loan can have multiple repayment records.
Customer → Investment	One customer can have multiple investments.
Account → Investment	Investment is linked to a funding account.
Investment → InvestmentPerformance	Performance snapshots are investment-linked and unique per investment/date.
Customer/User → FileMetadata	Customer documents are owned by a customer and associated with the uploading user.
User → Notification	Notifications belong to a user and use a unique event_id.
User → ApIdempotencyKey	Idempotency keys are unique per user/key pair.
User → RefreshToken	Refresh-token state is user-linked and supports token replacement.

12. REST API Reference

The implemented REST surface uses the /api/v1 namespace. The catalog below is derived from the REST controllers in the supplied source archive. It includes 84 documented operations; descriptions are concise functional summaries rather than generated sample payloads.

Module	Method & endpoint	Purpose
Auth	POST /api/v1/auth/login	Authenticate credentials and issue access/refresh tokens.
Auth	POST /api/v1/auth/change-password	Change authenticated user's password.
Auth	POST /api/v1/auth/refresh	Refresh an access token using a refresh token.
Auth	POST /api/v1/auth/logout	Logout/revokes refresh-token state.
Users	POST /api/v1/users	Create user.
Users	PUT /api/v1/users/{id}	Update user.
Users	GET /api/v1/users/{id}	Get user by ID.
Users	GET /api/v1/users/username/{username}	Get user by username.
Users	GET /api/v1/users	Get all users.
Users	DELETE /api/v1/users/{id}	Delete user.
Admins	POST /api/v1/admins	Create administrator.
Admins	PUT /api/v1/admins/{id}	Update administrator.
Admins	GET /api/v1/admins/{id}	Get administrator.
Admins	GET /api/v1/admins	Get administrators.
Admins	PATCH /api/v1/admins/{id}/deactivate	Deactivate administrator.
Employees	POST /api/v1/employees	Create employee.
Employees	PUT /api/v1/employees/{id}	Update employee.
Employees	GET /api/v1/employees/{id}	Get employee.
Employees	GET /api/v1/employees/user/{userId}	Get employee by user.
Employees	GET /api/v1/employees	Get employees.
Employees	DELETE /api/v1/employees/{id}	Delete employee.
Customers	POST /api/v1/customers	Create customer.
Customers	PUT /api/v1/customers/{id}	Update customer.
Customers	GET /api/v1/customers/{id}	Get customer.
Customers	GET /api/v1/customers/user/{userId}	Get customer by user.
Customers	GET /api/v1/customers	Get customers.
Customers	DELETE /api/v1/customers/{id}	Delete customer.
Customers	PATCH /api/v1/customers/{customerId}/deactivate	Deactivate customer.
Customers	PATCH /api/v1/customers/{customerId}/activate	Activate customer.
Accounts	POST /api/v1/accounts	Create account.
Accounts	GET /api/v1/accounts/{id}	Get account.
Accounts	GET /api/v1/accounts/customer/{customerId}	Get customer's accounts.

Module	Method & endpoint	Purpose
Accounts	GET /api/v1/accounts	Get accounts.
Accounts	PATCH /api/v1/accounts/{id}/block	Block account.
Accounts	PATCH /api/v1/accounts/{id}/activate	Activate account.
Accounts	PATCH /api/v1/accounts/{id}/deactivate	Deactivate account.
Accounts	PATCH /api/v1/accounts/{id}/close	Close account.
Transactions	POST /api/v1/transactions/deposit	Deposit funds.
Transactions	POST /api/v1/transactions/withdraw	Withdraw funds.
Transactions	POST /api/v1/transactions/transfer	Transfer funds and create transfer entries.
Transactions	GET /api/v1/transactions/{id}	Get transaction.
Transactions	GET /api/v1/transactions/account/{accountId}	Get account transactions.
Transactions	POST /api/v1/transactions/{id}/reverse	Reverse a transaction.
Loans	POST /api/v1/loans	Apply for a loan.
Loans	GET /api/v1/loans/{loanId}	Get loan.
Loans	GET /api/v1/loans/customer/{customerId}	Get customer's loans.
Loans	GET /api/v1/loans	Get loans.
Loans	GET /api/v1/loans/eligibility/{customerId}	Check loan eligibility.
Loans	PATCH /api/v1/loans/{loanId}/approve	Approve loan.
Loans	PATCH /api/v1/loans/{loanId}/reject	Reject loan.
Loans	PATCH /api/v1/loans/{loanId}/complete	Complete loan.
Loans	POST /api/v1/loans/{loanId}/disbursements	Disburse approved loan to an account.
Repayments	POST /api/v1/loan-repayments/loan/{loanId}	Make loan repayment.
Repayments	GET /api/v1/loan-repayments/{repaymentId}	Get repayment.
Repayments	GET /api/v1/loan-repayments/loan/{loanId}	Get repayments for a loan.
Repayments	GET /api/v1/loan-repayments	Get all repayments.
Investments	POST /api/v1/investments	Create investment.
Investments	GET /api/v1/investments/{investmentId}	Get investment.
Investments	GET /api/v1/investments/customer/{customerId}	Get customer's investments.
Investments	GET /api/v1/investments	Get investments.
Investments	PATCH /api/v1/investments/{investmentId}/mature	Mark investment as matured.
Investments	PATCH /api/v1/investments/{investmentId}/settle	Settle matured investment.
Performance	POST /api/v1/investment-performance	Record performance snapshot.
Performance	GET /api/v1/investment-performance/{performanceId}	Get performance snapshot.

Module	Method & endpoint	Purpose
Performance	GET /api/v1/investment-performance/investment/{investmentId}	Get performance history.
Performance	GET /api/v1/investment-performance/investment/{investmentId}/latest	Get latest performance.
Files	POST /api/v1/files	Upload customer document as multipart form data.
Files	GET /api/v1/files/{fileId}	Get file metadata.
Files	GET /api/v1/files/customer/{customerId}	Get customer files.
Files	GET /api/v1/files/{fileId}/download	Download stored file.
Files	DELETE /api/v1/files/{fileId}	Delete file metadata/storage.
Notifications	GET /api/v1/notifications	Get current user's notifications.
Notifications	GET /api/v1/notifications/unread	Get unread notifications.
Notifications	GET /api/v1/notifications/{notificationId}	Get current user's notification.
Notifications	PATCH /api/v1/notifications/{notificationId}/read	Mark notification as read.
Notifications	PATCH /api/v1/notifications/read-all	Mark all current user's notifications as read.
Audit	GET /api/v1/audit-logs	Search audit logs.
Audit	GET /api/v1/audit-logs/{id}	Get audit log.
Reports	POST /api/v1/reports/customers	Generate paged customer report.
Reports	POST /api/v1/reports/accounts	Generate paged account report.
Reports	GET /api/v1/reports/transactions	Generate paged transaction report.
Reports	GET /api/v1/reports/loans	Generate paged loan report.
Reports	GET /api/v1/reports/investments	Generate paged investment report.
Admin Accounts	PUT /api/v1/admin/accounts/{userId}/deactivate	Deactivate a user account administratively.

13. Validation and Error Handling

The project defines reusable validation infrastructure under `common.validation`, including `StrongPassword`, `ValidDateRange`, `StrongPasswordValidator`, `DateRangeValidator` and `ValidationUtil`. Request DTOs use Jakarta Validation annotations and controller methods apply `@Valid` where appropriate.

Exception Model

- `ApplicationException` as the application-level base exception pattern.
- `ResourceNotFoundException` for missing domain resources.
- `UnauthorizedException` and `ForbiddenException` for security/access failures.
- `BusinessException` for business-rule violations.
- `InvalidTransactionException` and `InsufficientBalanceException` for financial operation failures.
- `LoanEligibilityException` for loan eligibility failures.
- `IdempotencyException` for duplicate or invalid idempotency usage.
- `FileStorageException` for document-storage failures.
- `GlobalExceptionHandler` centralizes REST error conversion.

Common response types include `ApiResponse`, `ErrorResponse`, `FieldErrorResponse` and `PageResponse`. The application configuration deliberately suppresses message, binding-error details, stack traces and exception names from server error responses.

14. Transaction Integrity and Financial Controls

Financial operations are modeled as explicit service operations rather than generic CRUD. The transaction module provides deposit, withdrawal, transfer, transaction retrieval, reversal, loan-disbursement credit and investment-settlement credit operations.

Integrity mechanisms evidenced in the codebase

- `BigDecimal` is used for monetary entity fields and `DECIMAL(19,4)` is used by the primary financial schema.
- Account entities carry a version field for optimistic-locking support.
- Transaction references are unique in the database.
- Loan disbursement and investment settlement have dedicated transaction types and reference fields.
- Database `CHECK` constraints enforce positive amounts and valid status/type values.
- Transfer processing has dedicated transaction authorization and account authorization services.
- Idempotency protects mutating REST operations using a per-user key and request hash.

15. Idempotency Design

Mutating REST operations are protected by `IdempotencyAspect` and `IdempotencyKeyFilter`. The aspect requires an `Idempotency-Key` for mutating REST operations, derives the authenticated user identity from the JWT or `CustomUserDetails`, generates a request hash and delegates execution to `IdempotencyService`.

The `api_idempotency_keys` table stores the user, key, request hash, status, HTTP status, response body and lifecycle timestamps. The database enforces uniqueness on `(user_id, idempotency_key)`. Cleanup is scheduled using `IdempotencyCleanupScheduler`.

16. Auditing and Observability

Application Audit

AuditAspect surrounds public service-layer methods and publishes AuditEvent records containing actor identity, role, action classification, service/method names, entity context, HTTP request context and success/error information. AuditEventListener persists these records asynchronously after transaction completion.

JPA auditing is also configured through AuditableEntity and AuditorAwareImpl. Selected mutable entities carry createdBy and updatedBy fields.

Logging and Performance

- LoggingAspect records service-method start/completion/failure at appropriate log levels.
- PerformanceAspect measures service-method duration and emits warnings when execution exceeds bank.performance.slow-method-threshold-ms.
- Spring Boot Actuator exposes health, info and metrics endpoints.
- Production management configuration binds the management server to 127.0.0.1 and port 8081 by default, while the main application runs on port 8080 unless configured otherwise.

17. Caching

Caffeine is configured as the cache provider. Named caches include userById, userByUsername, customerById, customerById, employeeById, employeeById, accountById, accountByNumber, loanById and investmentById. Service implementations use @Cacheable and @CacheEvict around read/write operations.

The default application profile configures a one-minute expiry and maximum size of 1,000 entries. Production configuration makes cache size and expiry externally configurable.

18. Scheduling and Maintenance

Scheduler	Role	Configuration / cadence
IdempotencyCleanupScheduler	Scheduled task present	\${bank.scheduler.idempotency-cleanup-delay-ms:21600000}
InvestmentScheduler	Scheduled task present	\${bank.scheduler.investment-maturity-delay-ms:3600000}
LoanScheduler	Scheduled task present	\${bank.scheduler.loan-monitor-delay-ms:3600000}
NotificationScheduler	Scheduled task present	\${bank.scheduler.notification-maintenance-delay-ms:21600000}
RefreshTokenCleanupScheduler	Scheduled task present	\${bank.scheduler.refresh-token-cleanup-delay-ms:21600000}

SchedulingConfig enables scheduling. The configured scheduler areas cover investment maturity, loan monitoring, notification maintenance, refresh-token cleanup and idempotency cleanup. Notification and audit event listeners additionally use asynchronous execution with the bankTaskExecutor.

19. Notifications and Domain Events

Event	Consumer / purpose
Event	Consumer / purpose
CustomerCreatedEvent	Creates a CUSTOMER_CREATED notification after commit.
LoanApprovedEvent	Creates a LOAN_APPROVED notification after commit.
TransactionCompletedEvent	Creates a TRANSACTION_COMPLETED notification after commit.
InvestmentMaturityEvent	Creates an INVESTMENT_MATURED notification after commit.
AuditEvent	Persists an audit log asynchronously after transaction completion.

20. File and Document Management

The file module separates metadata persistence from physical storage. FileMetadata records the customer, uploading user, document type, original/stored names, storage path, content type, size, checksum and lifecycle status. LocalFileStorageService implements local storage.

The default configuration permits PDF, JPG/JPEG, PNG, XLSX and DOCX extensions and limits uploads to 2 MB in the base/dev/test configuration. Production configuration makes maximum file size, per-owner limits and storage root externally configurable.

The REST upload endpoint consumes multipart/form-data. File download returns a ByteArrayResource with content type and download filename determined by the file service.

21. Reporting

Reporting is implemented as a separate feature with ReportingService, ReportingRepository and ReportQuery abstractions. The API exposes paged reports for customers, accounts, transactions, loans and investments. ReportRequest supplies the reporting query parameters and response DTOs are specialized by report type.

The implementation uses Page-based responses for report endpoints, allowing the API to handle larger result sets without requiring the full dataset in a single response.

22. Configuration and Environments

Profile	Key behavior
application.yml	Base configuration; default profile dev; Flyway enabled; Caffeine cache; file limits; scheduler delays; API docs default private; Actuator health/info/metrics.
application-dev.yml	Local MySQL defaults, ddl-auto validate, debug logging, public API docs, JWT key environment variables, server port 8080.
application-test.yml	Dedicated test DB URL, random server port, reduced cache, temporary file storage, Swagger/OpenAPI disabled.
application-prod.yml	Externalized DB/JWT/file/cache settings, longer idempotency TTL, larger file defaults, separate management port/address, API docs disabled by default.

Secrets such as database credentials and JWT key material are externalized through environment variables in the profile configuration. The application does not embed production private-key material in the source configuration.

23. Database Migration History

Migration	Purpose
V1 create_users.sql	create users
V2 create_customers.sql	create customers
V3 create_employees.sql	create employees
V4 create_accounts.sql	create accounts
V5 create_transactions.sql	create transactions
V6 create_loans.sql	create loans
V7 create_loan_repayments.sql	create loan repayments
V8 create_investments.sql	create investments
V9 create_investment_performance.sql	create investment performance
V10 create_file_metadata.sql	create file metadata
V11 create_refresh_tokens.sql	create refresh tokens
V12 add_loan_disbursement_support.sql	add loan disbursement support
V13 add_customer_lifecycle.sql	add customer lifecycle
V14 create_notifications.sql	create notifications
V15 create_audit_logs.sql	create audit logs
V16 create_idempotency_keys.sql	create idempotency keys
V17 add_investment_settlement_support.sql	add investment settlement support
V18 correct_transaction_type_constraint.sql	correct transaction type constraint

The migration chain establishes the schema in incremental stages: identity → customers/employees → accounts → transactions → loans → repayments → investments → investment performance → file metadata → refresh tokens → loan disbursement support → customer lifecycle → notifications → auditing → idempotency → investment settlement support → transaction-type correction.

24. Testing Assets

Test source	Scope
src/test/java/com/example/bank/controller/user/UserRestControllerTest.java	Controller test
src/test/java/com/example/bank/service/user/UserServiceImplTest.java	Service test

The build also includes Spring Boot test starters for Actuator, Cache, Data JPA, Flyway, Security, Thymeleaf, Validation, plus Testcontainers MySQL/JUnit dependencies. The supplied source archive contains three explicit test classes; this document does not infer broader test coverage beyond those files and declared dependencies.

25. Build and Dependency Management

The project is Maven-based and includes the Maven Wrapper. The build uses spring-boot-maven-plugin with repackage and build-info goals. The maven-compiler-plugin is configured with MapStruct, Lombok and lombok-mapstruct-binding annotation processors.

The artifact coordinates in pom.xml are groupId com.example, artifactId bank and version 0.0.1-SNAPSHOT. The Spring Boot parent is 4.1.1 and Java 26 is configured through java.version.

26. API Documentation and Operational Interfaces

Springdoc is configured to scan REST controllers and match /api/v1/**. OpenAPI JSON is exposed at /v3/api-docs and Swagger UI at /swagger-ui.html when enabled. The base configuration keeps API documentation private; development enables it explicitly.

Actuator exposes health, info and metrics. Production places management endpoints on a separate management server address/port and exposes only the selected operational endpoints.

27. Professional Deployment Considerations

The project configuration supports a production profile with externalized database, JWT, file-storage, cache, idempotency and management settings. The application is designed to run with Flyway-managed schema validation and a stateless JWT REST API.

Operational checklist derived from the project configuration

- Provide a production MySQL database and DB_URL/DB_USERNAME/DB_PASSWORD.
- Provide Base64-encoded RSA JWT private/public keys through environment variables.
- Set a durable FILE_STORAGE_ROOT with appropriate filesystem permissions and backup strategy.
- Keep BANK_API_DOCS_PUBLIC false unless public API documentation is explicitly required.
- Review cache size/expiry and idempotency TTL for workload characteristics.
- Protect the management interface and retain the configured separate management binding.
- Run migrations before application traffic is admitted to the service.
- Configure log collection and monitoring for Actuator metrics and slow-method warnings.

28. Implementation Notes and Source Boundaries

This documentation is intentionally grounded in the supplied source archive. It describes implemented classes, endpoints, configuration and database migrations rather than presenting generic banking features as if they were implemented.

29. Conclusion

BankApplication is structured as a feature-oriented Spring Boot banking platform with a clear separation between API transport, application services, persistence, security and cross-cutting operational infrastructure. Its source tree covers core banking domains together with auditability, idempotency, event-driven notifications, scheduled maintenance, caching, reporting, document storage and JWT/session security.

The resulting architecture provides a coherent foundation for continued development and operational hardening. The most important implementation boundary visible in the supplied archive is the distinction between the extensive Thymeleaf resource set and the currently present MVC Java controller source; all other documented claims in this document are derived from the corresponding source/configuration/migration assets included in the archive.

Appendix A. Key Source Components

Area	Representative classes
Area	Representative classes
Application	com.example.bank.BankApplication
Security	SecurityConfig, JwtConfig, JwtTokenService, CustomUserDetailsService, authorization services
Cross-cutting	AuditAspect, IdempotencyAspect, LoggingAspect, PerformanceAspect, SecurityAuditAspect
Persistence	Spring Data repositories per business domain
Mapping	MapStruct mapper interfaces per DTO/entity domain
Events	CustomerCreatedEvent, LoanApprovedEvent, TransactionCompletedEvent, InvestmentMaturityEvent, AuditEvent
Listeners	NotificationEventListener, AuditEventListener
Scheduling	InvestmentScheduler, LoanScheduler, NotificationScheduler, RefreshTokenCleanupScheduler, IdempotencyCleanupScheduler
Operations	Actuator, OpenAPI/Springdoc, profile-based YAML configuration